PROGRAMAÇÃO FUNCIONAL

FUNCTORES (FUNCTORS)

CENTRO FEDERAL DE EDUCAÇÃO
TECNOLÓGICA DE MINAS GERAIS

MARCOS.EMILIANO@CEFETMG.BR
MARCOS EMILIANO

O CONCEITO DE FUNCTOR E A ABSTRAÇÃO DO MAP

Durante a disciplina, vimos como a função map :: (a -> b) -> [a] -> [b] nos permite **transformar** todos os **elementos** de uma **lista** sem **alterar** a **estrutura** da lista em si.

O CONCEITO DE FUNCTOR E A ABSTRAÇÃO DO MAP

No entanto, em programas funcionais **reais**, dados **frequentemente** residem dentro de outros **contêineres** ou **contextos** computacionais — como **Maybe a** (valores opcionais), **Either e a** (tratamento de erros) ou tipos **customizados** como árvores.

O CONCEITO DE FUNCTOR E A ABSTRAÇÃO DO MAP

Ter que **desembalar** e **reembalar** **manualmente** esses valores via pattern matching **toda vez** que desejamos aplicar uma transformação simples gera código **redundante** e **propenso** a erros.

O CONCEITO DE FUNCTOR E A ABSTRAÇÃO DO MAP

O Problema da Repetição:
Como **aplicar** uma **função** simples Double -> Double a um valor **envolvido** em **Just** 10.0 sem escrever **blocos case** ... of repetitivos?

O CONCEITO DE FUNCTOR E A ABSTRAÇÃO DO MAP

A Solução Abstrata:
O conceito de **Functor** permite **aplicar** uma **função** ao **conteúdo interno** de uma estrutura (contexto) **sem alterar** a estrutura que **envolve** esse valor.

O CONCEITO DE FUNCTOR E A ABSTRAÇÃO DO MAP

Intuição do Contêiner:
Pense no **Functor** como uma "caixa" ou "contexto". O mapeamento aplica a **transformação** no objeto **dentro da caixa** sem quebrar a caixa.

A TYPECLASS FUNCTOR E A FUNÇÃO FMAP

Em Haskell, a **classe** de tipos **Functor** é definida no módulo Data.Functor (ou Prelude) da seguinte forma:

```
1. class Functor f where
2.     fmap :: (a -> b) -> f a -> f b
```

A TYPECLASS FUNCTOR E A FUNÇÃO FMAP

Análise de Tipos (Kind):
O parâmetro **f** na definição **não** representa um **tipo concreto** (como Int ou Bool), mas sim um **construtor** de tipo de ordem superior (com kind * -> *), como [], Maybe ou IO.

A TYPECLASS FUNCTOR E A FUNÇÃO FMAP

O Operador Infixo <$> :

A biblioteca padrão fornece o operador <**$**> como um **alias infixo** para **fmap**, facilitando a leitura de **chamadas sequenciais**:

```
(<$>) :: Functor f => (a -> b) -> f a -> f b
(<$>) = fmap
```

NOTAÇÕES DE OPERADORES

Notação Prefixa (Padrão Funcional):

- O **operador** ou nome da função **precede** seus **operandos**: função arg1 arg2.

Qualquer operador infixo binário torna-se uma função prefixa quando **envolvido** por **parênteses**.

(+) 5 3 é equivalente a 5 + 3.

NOTAÇÕES DE OPERADORES

Notação Infixa (Entre Operandos):

- O operador é posicionado **entre** dois argumentos: arg1 op arg2 (comum para operadores como +, *, <**$**>).

Qualquer função **binária** prefixa pode ser utilizada como infixa ao envolver seu nome em **crases** (backticks).

10 `div`2 é equivalente a div 10 2.

A TYPECLASS FUNCTOR E A FUNÇÃO FMAP

O operador <**$**> é apenas um atalho visual para a função fmap. Ele existe para deixar a **leitura** do código **mais natural**, imitando a forma como já aplicamos **funções normais** em Haskell.

A TYPECLASS FUNCTOR E A FUNÇÃO FMAP

Para entender a analogia:

- Aplicação comum (**$**): f **$** x significa "aplique a função f no valor x".
- Aplicação em Functor (<**$**>): f <**$**> x significa "aplique a função f no valor que está dentro do **recipiente** x".

A TYPECLASS FUNCTOR E A FUNÇÃO FMAP

```
-- Usando fmap (a função é chamada antes de tudo)
fmap (+10) (Just 5)   -- Resultado: Just 15

-- Usando <$> (o operador fica no meio, entre a função e a caixa)
(+10) <$> Just 5      -- Resultado: Just 15
```

DESMISTIFICANDO LAMBDAS

Aplicação Parcial Inerente (Currying):

- Em Haskell, todas as funções aceitam estritamente **um parâmetro** por vez.
- Fornecer **menos argumentos** do que o esperado **não gera** um erro, mas sim uma **nova função** que **aguarda** os **argumentos restantes**.

Consequência: \x -> soma 5 x é idêntico a simplesmente soma 5.

DESMISTIFICANDO LAMBDAS

Seções de Operadores (Operator Sections):

- Operadores **infixos** podem ser **parcialmente aplicados** envolvendo-os entre parênteses com apenas um dos operandos.

Consequência: \x -> x + 1 simplifica-se para (+ 1).

INSTÂNCIAS CLÁSSICAS: LISTAS E MAYBE

Instância para Listas ([]):
O mapeamento em listas é simplesmente a função map tradicional.

```
instance Functor [] where
    fmap = map
```

INSTÂNCIAS CLÁSSICAS: LISTAS E MAYBE

Instância para Maybe:
Preserva o **contexto** de **ausência** de valor (Nothing) ou **aplica** a **função** ao valor existente (Just x).

```
instance Functor Maybe where
    fmap _ Nothing  = Nothing
    fmap f (Just x) = Just (f x)
```

INSTÂNCIAS CLÁSSICAS: LISTAS E MAYBE

Exemplos no REPL (GHCi):

```
> fmap (*2) [1, 2, 3]
[2, 4, 6]

> fmap (+5) (Just 10)
Just 15

> (+5) <$> Nothing
Nothing
```

INSTÂNCIA EM TIPOS RECURSIVOS CUSTOMIZADOS (LISTAS)

Já vimos que a **lista nativa** do Haskell ([]) é uma **instância** de **Functor**. No entanto, para compreender o funcionamento do fmap em **estruturas recursivas** sem depender das facilidades sintáticas da linguagem (como a notação [1, 2, 3]), é interessante implementar a instância de **Functor** para um tipo recursivo linear: a **Lista Encadeada**.

INSTÂNCIA EM TIPOS RECURSIVOS CUSTOMIZADOS (LISTAS)

Considere a definição de uma **lista encadeada** customizada utilizando **construtores** de dados **explícitos**:

```
-- Tipo Algébrico de Dados Recursivo
data Lista a = Nulo
             | Cons a (Lista a)
             deriving (Show, Eq)
```

INSTÂNCIA EM TIPOS RECURSIVOS CUSTOMIZADOS (LISTAS)

A implementação da função **fmap** percorre a **sequência de construtores** aninhados aplicando a função **f** a cada elemento do tipo a, preservando a estrutura da lista intacta até atingir o caso base (Nulo).

```
instance Functor Lista where
    fmap _ Nulo        = Nulo
    fmap f (Cons x xs) = Cons (f x) (fmap f xs)
```

INSTÂNCIA EM TIPOS RECURSIVOS CUSTOMIZADOS (LISTAS)

No padrão **Cons x xs**, o uso dos **parênteses** é **obrigatório** para **desestruturar** o construtor **Cons** em seus **dois componentes**: a cabeça x (do tipo a) e a cauda xs (do tipo Lista a).

A **chamada recursiva** fmap f xs é aplicada diretamente sobre a **cauda**, reconstruindo a lista com o mesmo construtor Cons.

INSTÂNCIA EM TIPOS RECURSIVOS CUSTOMIZADOS (LISTAS)

Simulação no REPL (GHCi):

```
-- Construção de uma lista: 1 : 2 : 3 : []
> let minhaLista = Cons 1 (Cons 2 (Cons 3 Nulo))

-- Mapeamento aplicando multiplicação por 10
> fmap (*10) minhaLista
Cons 10 (Cons 20 (Cons 30 Nulo))

-- Utilizando o operador infixo <$>
> (+5) <$> minhaLista
Cons 6 (Cons 7 (Cons 8 Nulo))
```

INSTÂNCIA EM TIPOS RECURSIVOS CUSTOMIZADOS (ÁRVORES)

Considere a seguinte estrutura de dados recursiva para uma árvore binária:

```
data Arvore a = Folha
              | No a (Arvore a) (Arvore a)
              deriving (Show, Eq)
```

INSTÂNCIA EM TIPOS RECURSIVOS CUSTOMIZADOS (ÁRVORES)

Considere a seguinte estrutura de dados recursiva para uma árvore binária:

```
instance Functor Arvore where
    fmap _ Folha        = Folha
    fmap f (No x esq dir) = No (f x) (fmap f esq) (fmap f dir)
```

INSTÂNCIA EM TIPOS RECURSIVOS CUSTOMIZADOS (ÁRVORES)

Propriedade de Preservação:
Ao aplicar **fmap f** em uma Arvore, a forma gráfica da árvore (**número** de nós, folhas e ramificações) é **estritamente preservada**; apenas os **dados** contidos nos nós **sofrem alteração**.

INSTÂNCIA EM TIPOS RECURSIVOS CUSTOMIZADOS (ÁRVORES)

Simulação no REPL:

```
> let minhaArvore = No 10 (No 5 Folha Folha)
(No 20 Folha Folha)
> fmap (* 3) minhaArvore
No 30 (No 15 Folha Folha) (No 60 Folha Folha)
```

LEIS DOS FUNCTORES (FUNCTOR LAWS)

Toda instância válida de **Functor** deve obrigatoriamente **satisfazer duas leis** fundamentais.

LEIS DOS FUNCTORES (FUNCTOR LAWS)

Primeira Lei: Preservação da Identidade
Mapear a função **identidade** (id) sobre um **functor** deve retornar o functor **inalterado**.

```
fmap id = id
```

```
> fmap id (Just 42)
Just 42
```

LEIS DOS FUNCTORES (FUNCTOR LAWS)

Segunda Lei: Preservação da Composição
Mapear a **composição** de **duas funções** (f . g) **equivale** a aplicar **fmap g** e, em seguida, **fmap f**.

$$\text{fmap } f \circ g = \text{fmap } f \circ \text{fmap } g$$

LEIS DOS FUNCTORES (FUNCTOR LAWS)

```
> let f = (+2)
> let g = (*3)

> fmap (f . g) (Just 5)
> Just 17

> (fmap f . fmap g) (Just 5)
> Just 17
```

LEIS DOS FUNCTORES (FUNCTOR LAWS)

Importância Prática:
A **Lei da Composição** permite que o **compilador** realize **otimizações** conhecidas como fusion (fusão de loops), **combinando** múltiplos **Mapeamentos** encadeados em um **único passo** de execução **sem percorrer** a estrutura **repetidamente**.

DESAFIOS PRÁTICOS

Dada a lista de valores opcionais amostras :: [Maybe Int], implemente a função incrementarOpcoes :: [Maybe Int] -> [Maybe Int] que incrementa em 1 unidade cada valor presente dentro dos construtores Just, mantendo os valores Nothing inalterados. Utilize o operador infixo <S>.

DESAFIOS PRÁTICOS

Dado o tipo de dados data RegistroCliente a = Cliente String a (onde o primeiro campo é o nome do cliente e o segundo representa um dado genérico a), declare a assinatura da instância de Functor para RegistroCliente e implemente a função fmap correspondente, de modo que a transformação afete apenas o segundo campo.

DESAFIOS PRÁTICOS

Um sistema de e-commerce armazena o preço final de produtos em uma estrutura data PrecoProduto a = PrecoBase a | Promocao a Double (onde Promocao guarda o preço original a e a porcentagem de desconto Double).

1. Crie a instância de Functor para PrecoProduto.
2. Implemente a função aplicarTaxaCâmbio :: Double -> PrecoProduto Double -> PrecoProduto Double que multiplica o preço base/original pela taxa informada sem alterar a porcentagem de desconto.

DESAFIOS PRÁTICOS

Considere o tipo recursivo de lista encadeada customizada data Sequencia a = Vazia | Item a (Sequencia a) deriving (Show, Eq).

1. Implemente a instância de Functor para Sequencia.
2. Escreva a função normalizarSequencia :: Floating a => Sequencia a -> Sequencia a que aplica a raiz quadrada (sqrt) a todos os elementos contidos na sequência.

DÚVIDAS???